My chief of staff, Claude Code

Jim Prosser @jimprosser · 05/03/2026

https://x.com/jimprosser/status/2029699731539255640

The dream of modern personalized technology has always been some kind of perfect digital assistant—a chief of staff that triages your inbox, organizes your day, and handles the operational overhead of your life before you've had your first coffee. For most people, that dream still lives in the vague future, somewhere between "Siri, but good" and science fiction.

It's achievable today, even for people who aren't programmers. I know because I just built it over 36 hours with Claude Code.

I should be specific about my coding credentials here: I have none. I'm a 43-year-old tech communications consultant who runs a solo practice in Marin County, California. But I designed this system, and the system works because Claude Code lets me think and then takes care of the rest.

What It Actually Does

Every morning before I wake up, two automated processes run on my always-on Mac Studio. The first scans the present and next day's Google Calendar for meetings at physical locations, calculates real drive times using the Google Maps API, and creates transit time events so I know when to leave and don't double-book anything. The second triages yesterday's email, identifies anything requiring action, checks my task manager (I use Todoist) for duplicates, and creates properly attributed tasks with priorities, due dates, and duration estimates.

By 6:15 AM, my task system is current without me touching it.

When I get to my desk, I hit a button on my Stream Deck labeled "AM Sweep." This triggers a command in Claude Code that pulls my tasks along with my calendar and recent meeting transcripts for context, then classifies every item into one of four categories. Green means Claude can complete the task fully on its own. Yellow means it can get 80% of the way there and I finish. Red means it needs my brain or my presence. Gray means it's not actionable today. I see the full categorized list, adjust anything that looks off, and say "go."

Here's where it gets interesting. Six specialized AI agents fire in parallel, each with their own context window and scoped tool access. One drafts emails (it never sends, only drafts). Another updates my client files in Obsidian, my knowledge management system. Another schedules meetings. Another runs background research on a prospect, a topic, or a news event. They all work simultaneously while I focus on strategic work: writing, calls, the stuff that actually requires my brain and judgment.

A couple minutes later, I get a completion report. Tasks marked done. Email drafts sitting in Gmail for my review. Client notes updated. Research filed. Next steps flagged.

```
● ☀️ Morning Sweep — Thursday, March 6, 2026

Today's Calendar:
- 10:00–10:30 AM — Weekly sync w/ Sarah Chen (Meridian Health) — Zoom
- 11:00–11:45 AM — Discovery call w/ Apex Robotics (Marcus Webb, CEO) — Zoom
- 12:30–1:15 PM — ☕ Coffee w/ David Kwon (Redpoint) — Equator Coffee, Mill Valley
- Transit → Equator Coffee (12:15–12:30 PM)
- Transit → Office (1:15–1:30 PM)
- 3:00–3:30 PM — Internal: review Q1 pipeline numbers

🟢 DISPATCH — Ready to Execute (5 tasks):

1. Reply to Sarah Chen re: revised messaging framework delivery timeline (Meridian Health)
Agent: comms-agent | Tools: Gmail, Granola
Plan: Pull yesterday's Granola transcript from the Meridian check-in where she asked about the revised timeline. Draft reply confirming Friday delivery, note the brand voice section is what's taking the extra day.
Context source: Granola transcript (Mar 5), task description

2. Research Marcus Webb + Apex Robotics ahead of 11 AM discovery call (BizDev)
Agent: research-agent | Tools: Web search, Filesystem
Plan: Background on Webb (career, LinkedIn, recent interviews), Apex Robotics (product, funding, go-to-market), and current moment (Series B announced last week). Save to Biz Dev/Apex-Robotics/Meeting Prep — Marcus Webb 2026-03-06.md.
Context source: Calendar event details, Todoist description (includes referral context from Jamie at Lux)

3. Update Meridian Health Strategic Brief with Q1 earned media results (Meridian Health)
Agent: knowledge-agent | Tools: Filesystem, Granola
Plan: Pull the coverage metrics Sarah shared in yesterday's sync (Granola transcript) and add to the Results & Traction section of the Strategic Brief. Three placements landed: Protocol, The Information, Axios.
Context source: Granola transcript (Mar 5), existing Strategic Brief

4. Send David Kwon a calendar confirm + location for coffee (BizDev)
Agent: comms-agent | Tools: Gmail, Calendar
Plan: Draft short confirmation email — Equator Coffee, 12:30 PM, looking forward to catching up. He introduced the Apex Robotics lead so mention looking forward to discussing how that conversation goes.
Context source: Calendar event, Gmail thread (Kwon intro to Webb, Feb 22)

5. Archive Luma Diagnostics to xx-Archive (Ops)
Agent: knowledge-agent | Tools: Filesystem, Todoist
Plan: Engagement ended last week and closeout doc is complete. Move Clients/Luma-Diagnostics/ to Clients/xx-Archive/2026-Q1/Luma-Diagnostics/. Update Client Hub frontmatter to status: archived.
Context source: Todoist description, existing closeout doc

🟡 PREP — Can Get Ready for You (3 tasks):

6. Draft Apex Robotics follow-up email after discovery call (BizDev)
Agent: comms-agent | What I'll prepare: Two draft versions — one assuming strong fit (proposing a diagnostic engagement memo), one assuming exploratory (suggesting a second call with their VP Comms). Will incorporate whatever #2's research surfaces.
What you'll need to do: Pick the right draft after the call, adjust tone, send.

7. Draft engagement memo for Conduit Financial (BizDev)
Agent: comms-agent | What I'll prepare: Pull Granola transcript from Monday's second discovery call + your brain dump, prospect note, Gmail thread. Draft the diagnostic engagement memo using the Engagement Memo Template and Voice Guide. Will check Pricing Framework for comparable scoping.
What you'll need to do: Review diagnosis and pricing, edit for voice, send.

8. Compile Q1 pipeline summary for internal review at 3 PM (Ops)
Agent: research-agent | What I'll prepare: Pull all prospect notes from Biz Dev/, summarize each by stage, win probability, and estimated revenue. Format as a quick-reference table.
What you'll need to do: Add your gut-feel annotations on likelihood before the 3 PM review.

🔴 YOURS — Needs You (2 tasks):

9. Discovery call with Marcus Webb / Apex Robotics (BizDev)
Why: Live call — this is the introduction, relationship starts here. Research from #2 will be ready before 11 AM.
Anything I can prep? Already handled — see #2 (research) and #6 (post-call follow-up draft).

10. Write Strategic Brief for Conduit Financial (Conduit Financial)
Why: Strategic Brief is 100% you, no AI. Engagement memo (#7) feeds into this but the strategic interpretation is yours.
Anything I can prep? #7 gets the memo draft ready, which gives you a running start.

⚪ SKIP — Suggest Deferring (1 task):

11. Explore podcast booking strategy for Meridian CEO — Meridian hasn't approved the exec visibility workstream yet. Revisit after next week's steering committee. Suggest rolling to Mar 12.

---
Ready to dispatch? Tell me:
- "go" — dispatch all green and yellow tasks
- "go green" — dispatch only green tasks
- Or cherry-pick: "go 1, 3, 5" (by number)
- Or adjust: "move 2 to yellow" / "skip 4"
```

The AM Sweep in action. Fictional tasks and company names, but real system format and logic.

Then I press a second button on the Stream Deck called "Time Block" that executes another command that turns my remaining Todoist tasks into a time-blocked calendar using the duration estimates already assigned above. It knows which tasks need to happen at home versus the office versus specific locations. It batches all my errands into a single outing, routed geographically to minimize backtracking, with real Google Maps drive times. It schedules gym time on the right days. It puts home tasks in the evening window after my kids are in bed. If something doesn't fit today, it recommends a specific future day based on how loaded that day already is.

I review the proposed schedule, adjust if needed, and say "go." Calendar events appear. My day is structured. One minute.

The first time I watched the whole thing work, with six agents running in parallel, my terminal filling with progress updates, I just stared at the screen and laughed. Now I just jam to some music and start cracking away at tasks.

The Architecture (Not the Code)

The entire system runs on Claude Code. If you haven't used it: you describe what you want in plain English, and Claude writes and executes the code. The key capability that makes my system possible is subagents, basically independent AI workers that Claude Code spins up in parallel, each with their own instructions, tool access, and context.

But the code isn't the interesting part. The interesting part is the design.

The critical insight was figuring out the layering: each component had to know the others existed. The overnight email scanner doesn't just dump tasks into my task manager; it attributes them with the metadata that the Morning Sweep needs to classify them correctly. The AM Sweep doesn't just triage; it assembles context packages that each subagent needs to do its job in an independent context window. The time-blocker doesn't just schedule; it reads the output of everything upstream and accounts for it. You get the picture.

This is what separates a *system* from a collection of scripts. Each piece is designed to feed the next one. Remove any layer and the others still work. But the whole is dramatically more than the sum: six independent workers holding different contexts simultaneously, none of them competing for attention or memory like talking to a chatbot.

The actual building was a conversation. I wrote detailed Markdown files describing exactly how I wanted each piece to behave: classification criteria, voice guidelines, scheduling rules, tool permissions. Claude Code read those plain-English files as instructions and implemented them. I iterated by running the system against my real task list and tuning when things got miscategorized or an agent made a bad judgment call. Thirty-six hours, start to finish.

I didn't need to understand the code at a syntax level at all. But I did need to have a clear picture of the *architecture*: what talks to what, what each piece is responsible for, where the human-AI boundaries are. That's systems thinking, not software engineering.

The Most Crucial Design Principle

The single most important decision I made was being extremely deliberate about what stays human.

The system never sends an email. It drafts and I review. The system never writes my Strategic Briefs, the core strategic documents that define each client engagement. Those are 100% mine. The system never makes pricing decisions, handles relationship-sensitive communications, or makes strategic calls. When it's uncertain whether a task needs my judgment, it defaults to "prep" (get it 80% ready for me) rather than "dispatch" (handle it fully).

This sounds obvious, but experience tells me otherwise. It's probably the decision most people most frequently get wrong when they build automations like this or start thinking about "agentifying" their work. They either go too timid, and the system is basically a fancy to-do list that doesn't actually *do* anything, or too aggressive, and the AI starts firing off emails that don't sound like you and making decisions you didn't authorize. The middle ground is narrow and specific to your operation, and finding it is the real work.

The framework I landed on: dispatch, prep, yours, skip. Green tasks get fully handled. Yellow tasks get prepped with options for me to choose from. Red tasks are flagged as mine with whatever supporting context the system can assemble. Gray tasks get deferred with a reason. The system's bias is always toward involving me on anything ambiguous.

What It Replaced

I run a boutique consultancy. Five clients maximum, all referral, no junior staff. The system replaces what would otherwise be a skilled operations person, someone who knows my clients, understands my voice, manages my calendar, and handles the daily operational overhead of running a professional services firm.

Before this, that person was... me. Every morning started with 30-45 minutes of reviewing email, updating tasks, scheduling, filing notes, and doing the kind of operational housekeeping that's necessary but not valuable. That time is now single-digit minutes.

So the automations are saving me 30-45 minutes every morning, five days a week, across a year, to say nothing of the time saved from the tasks Claude can handle straightaway. That's somewhere between 130 and 195 hours annually on its own. Not exactly a rounding error.

The bigger win isn't time, it's cognitive load on my brain that's saved. When I sit down in the morning, the operational state of my business is already organized, triaged, and waiting for decisions rather than assembly. I start the day in decision mode instead of gathering mode. If you've ever run a business by yourself, you know that distinction isn't abstract. It's the difference between a morning that drains you and one that focuses you. The overnight automations also killed a low-grade anxiety about email I'd been carrying so long I didn't notice it until it was gone.

What It Costs

I'm on Claude's Max plan, which is $100/month as of this writing. Claude Code is included in the subscription, though heavy usage can exceed what's covered. In practice, most of my usage fits within the plan. When it doesn't, the overrun is trivial and laughably small relative to the productivity gains. The overnight automations use the cheaper, faster Sonnet model while everything else uses the more robust logic of the Opus model. My Google Maps API calls are well under Google's cap for the free tier. Hosting is free, since everything runs locally on my Mac Studio.

Total incremental cost beyond the subscription I was already paying: *maybe* $5-10/month on the high end. A part-time virtual assistant doing comparable work would cost somewhere between $400/month on the low end and $1,000/month on the high end, and they wouldn't have instant access to (or near-instant processing of) my meeting transcripts, email history, and knowledge base.

The Bigger Point

I didn't set out to build a chief of staff straightaway. I set out to solve one problem, and solving it revealed the next one. The overnight inbox scan made the morning triage better. Better triage made the subagent dispatch possible. Reliable dispatch made time-blocking viable. Each piece assumed the others existed. Thirty-six hours of work, compounding on itself.

If you're in San Francisco or the Valley, you might think everyone's doing this already. They're not. Step outside the tech bubble and almost nobody is using AI to build the operational infrastructure of their actual business. That gap is going to close eventually, and when it does, the professionals who figured it out early and developed the instinct for what to automate and

what to keep human, who learned to think in systems rather than prompts, are going to have an enormous advantage over the ones who waited for someone to package it into a SaaS product.

I don't know exactly where this goes next. But I know that my mornings are pretty great now, and the difference isn't just efficiency. It's that the operational weight of running a business has been redistributed in a way that lets me spend my time on the things I actually want to do.

(Oh, and the chief of staff is just the morning operations layer piece. I also built systems that automates my entire client lifecycle paired with a holistic knowledge architecture that gives Claude persistent context about my entire business. But that's a post for another day...)